

P2929R0

Proposal to add `simd_invoke` to `std::simd`

Published Proposal, 2025-06-17

This version:

<http://wg21.link/P2929R0>

Authors:

[Daniel Towner](#) (Intel)

[Ruslan Arutyunyan](#) (Intel)

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

Proposal to extend `std::simd` with a method of allowing a lambda to be invoked on smaller pieces of a SIMD value in order to make interaction with intrinsics easier.

Table of Contents

1	Motivation
2	Background
3	<code>simd_invoke</code> description
3.1	Using indexed Callable invocations
3.1.1	Design option - probing the index capabilities
3.2	Considerations in using <code>simd_invoke</code>
4	Wording
4.1	Add new section [<code>simd.invoke</code>]
	References
	Informative References

§ 1. Motivation

ISO/IEC 19570:2018 introduced data-parallel types to the C++ Extensions for Parallelism TS [P1928R3]. That paper, and several ancillary papers, do an excellent job of setting out the main features of an extension to C++ which allows generic data parallel programming on arbitrary targets. However, it is inevitable that the programmer will want to make some use of target-specific intrinsics in order to unlock some of the more unusual features of those specific platforms. This requires that the programmer is able to allow a SIMD value to be used in a call to a target intrinsic, and that the result of the intrinsic call can be used to generate a new SIMD value. This is already permitted in `std::simd` for SIMD values which fit into a native register, but it is harder to achieve when the SIMD value spans multiple registers.

In this paper we will propose a function called `simd_invoke` which makes it easy to repeatedly apply a target specific intrinsic to native-sized pieces of large SIMD value arguments, and to marshall their individual results back into a SIMD result.

§

Although `std::simd` has been carefully crafted to include APIs which access all of the common or desirable features of SIMD instruction sets, it is inevitable that the user will sometimes want to take advantage of instructions which are specific to a particular platform. For example, a DSP target may have a special type of accumulator instruction, or algorithm specific instruction (e.g., AES crypto). Clearly, calling these intrinsics results in non-portable code, but the increase in hardware-accelerated performance on a given target could be a worthwhile trade-off.

`std::simd` already includes some provision for interacting with target-specific data types by providing the following ([P1928R4] 28.9.6.1-4):

```
constexpr explicit operator implementation-defined() const;
constexpr explicit basic_simd(const implementation-defined& init);
```

These allow a SIMD value to be converted into a target specific type which is used to call a compiler-intrinsic. The result is then converted back into the equivalent SIMD type:

```

simd<float>
addsub(simd<float> a, simd<float> b) {
    return static_cast<simd<float>>(_mm256_addsub_ps(static_cast<__m256>(a), static_cast<__m256>(b)));
}

```

In this example the inputs, which are native register-sized SIMD values, are explicitly converted into their target-specific typed values. Those target specific types are used to call the intrinsics, and then the target specific return value is converted back into the closest SIMD representation.

This example is straightforward, since the SIMD values are explicitly the correct native size. For targets which supported several different register sizes (e.g., some variants of AVX support 128-, 256- and 512-bit registers) the code can use a `constexpr` conditional to select which size to use:

```

simd<float>
addsub(simd<float> a, simd<float> b) {
    if constexpr (simd<float>::size() == 4)
        return simd<float>(_mm_addsub_ps(static_cast<__m128>(a), static_cast<__m128>(b)));
    else if constexpr (simd<float>::size() == 8)
        return simd<float>(_mm256_addsub_ps(static_cast<__m256>(a), static_cast<__m256>(b)));
    else
        error(); // Invalid native register
}

```

Things become more tricky when dealing with SIMD values which are larger than their intrinsic types. Such types cannot be converted into a type which can be used to call an intrinsic. Instead, the big SIMD must be broken down into small pieces which may be used to call the intrinsic, and then the results of that glued back together. Here is one way to do that for a SIMD value which is twice as big as a native register:

[illegible]

```
// Glue the individual results back together.
return simd_concat(resultLow, resultHigh);
}
```

This is now getting verbose, and it only handles SIMD value inputs which are twice the size of a register value. To use the intrinsic with larger SIMD values, or SIMD value which don't map into native register-sized pieces, more work is needed (e.g., if `simd<float, 20>` was used then the pieces would be of size 8, 8, and 4 respectively, and this would require a suitable call to the intrinsic of the appropriate size).

The boiler-plate code needed to handle this is technical straight-forward, but verbose. A completely generic solution which could handle arbitrary SIMD value size would also requires additional mechanisms like immediately invoked lambdas or index sequences to be used too. Rather than requiring every user of `std::simd` to have to write their own intrinsic call handlers, we can abstract the general mechanism into something that is easily reused. In particular we want to break a set of SIMD value arguments into smaller pieces, call an intrinsic on each, and then glue the results of those intrinsic calls back together. We achieve this through a proposed function called `simd_invoke` which we will describe in the remainder of this paper.

3. `simd_invoke` description

The `simd_invoke` function is rather like `std::invoke` in that it takes a callable object and a set of arguments. Its basic signature is as follows:

```
template<typename Fn, typename... Args>
auto simd_invoke(Fn&& fn, Args&&...);
```

The `fn` parameter should be a callable that accepts some arguments which can be used to invoke an intrinsic from a native register. For example, to continue our example from above, we could create a utility function which calls the `_mm256_addsub_ps` intrinsic from native-sized SIMD values:

```
inline auto native_addsub(simd<float> lhs, simd<float> rhs) {
    auto nativeLhs = static_cast<__m256>(lhs);
    auto nativeRhs = static_cast<__m256>(rhs);

    return simd<float>(_mm256_addsub_ps(nativeLhs, nativeRhs));
}
```

Given this wrapper for native-sized intrinsic calls, we can now use `simd_invoke` to break down a large SIMD value into individual register-sized calls:

```
auto addsub(simd<float, 32> x, simd<float, 32> y)
{
    return simd_invoke(native_addsub, x, y);
}
```

The `simd_invoke` function accepts any number of arguments and will break each one down into native-sized pieces which are used to invoke the supplied function, and then glue the results back together again on completion.

Let's look how the example from [§2 Background](#) looks like with `simd_invoke` applied using before-after table:

Before	After
<pre>auto addsub(simd<float, 16> a, simd<float, 16> b) { auto [lowA, highA] = simd_split<simd<float>>(a); auto [lowB, highB] = simd_split<simd<float>>(b); auto resultLow = simd<float>(_mm256_addsub_ps(static_cast<__m256>(lowA), static_cast<__m256>(lowB)));</pre>	<pre>auto addsub(simd<float, 16> a, simd<float, 16> b) { { auto do_native = [] (simd<float> lhs, simd<float> rhs) { return simd<float>(_mm256_addsub_ps(static_cast<__m256>(lhs), static_cast<__m256>(rhs))); }; }</pre>

```

    auto resultHigh = simd<float>(_mm256_addsub_ps(
        static_cast<__m256>(highA),
        static_cast<__m256>(highB)));

    return simd_concat(resultLow, resultHigh);
}

```

```

    return simd_invoke(do_native, x, y);
}

```

By default the function will use the native size for the element type, with the aim of calling the intrinsic with the largest permitted builtin type. However, `simd_invoke` can also be explicitly given the size of block to use. For example, the following code breaks down into pieces of size 4 instead (and also supplies the callable as a local lambda, to make the function self-contained):

```

auto addsub(simd<float, 32> x, simd<float, 32> y)
{
    auto do_native = [] (simd<float, 4> lhs, simd<float, 4> rhs) {
        return simd<float, 4>(_mm_addsub_ps(static_cast<__m128>(lhs), static_cast<__m128>(rhs)));
    };

    return simd_invoke<4>(do_native, x, y);
}

```

Being able to define a different size is useful for two reasons:

1. We may wish to process data in smaller input sizes than native size. For example, if the data needs to be upcast to a large element size for the operation, then it can be useful to choose a smaller block size to begin with so that the upcast data fills a native register. This could be more efficient than accepting native register sized data and then upcasting to several registers.
2. If the element type of the arguments are different then the `simd_invoke` function cannot determine the appropriate native size for itself. For example, the first argument might be a SIMD value containing float elements, and the second argument have `uint8_t` elements. The number of native elements in each argument cannot be inferred, and the `simd_invoke` must be told how many elements to use in each block.

So far we have only considered what happens when `simd_invoke` is given SIMD value arguments which are multiples of some native register size, but as we saw in our introduction, it might be useful to be able to call invoke on SIMD values of arbitrary size. For example, suppose that the add/sub is being called on a SIMD value with 19 elements. In that case on a target with a native size of 8 elements `simd_invoke` would need to break the calls down into pieces of sizes 8, 8, and 3 respectively, and the callable would need to be able to handle SIMD values of arbitrary size. The following example shows how this might work:

```

auto addsub(simd<float, 19> x, simd<float, 19> y)
{
    // Invoke the most appropriate intrinsic for the given simd types.
    auto do_native =
        [<typename T, typename ABI>(basic_simd<T, ABI> lhs, basic_simd<T, ABI> rhs) {
            constexpr auto size = basic_simd<T, ABI>::size;
            if constexpr (size <= 4)
                return simd<float, size>(_mm_addsub_ps(static_cast<__m128>(lhs),
                    static_cast<__m128>(rhs)));
            else
                return simd<float, size>(_mm256_addsub_ps(static_cast<__m256>(lhs),
                    static_cast<__m256>(rhs)));
        }];

    return simd_invoke(do_native, x, y);
}

```

In this example the local lambda function can accept SIMD inputs of any size, and will choose the most appropriate intrinsic to use. For example, given the block of 3 tail elements the lambda utility will convert the 3 elements to an `__m128` register and call `_mm_addsub_ps`. This lambda function can then be called by `simd_invoke` to enable a completely arbitrarily

sized set of SIMD arguments to be mapped onto their underlying intrinsics. For the example above, the following code was generated:

```
vmovups ymm0, ymmword ptr [rsi]
vmovups ymm1, ymmword ptr [rsi + 32]
vmovups xmm2, xmmword ptr [rsi + 64]
mov     rax, rdi
vaddsubps ymm0, ymm0, ymmword ptr [rdx]
vaddsubps ymm1, ymm1, ymmword ptr [rdx + 32]
vaddsubps xmm2, xmm2, xmmword ptr [rdx + 64]
vmovups ymmword ptr [rdi], ymm0
vmovups ymmword ptr [rdi + 32], ymm1
vmovups xmmword ptr [rdi + 64], xmm2
```

Notice how the the load, addsub and store instructions work respectively in `ymm`, `ymm` and `xmm` registers to cope with the different sizes.

Having now described the basic operation of `simd_invoke` we can consider some of the rules for using it, and a useful extension to it which makes certain scenarios easier to deal with.

3.1. Using indexed Callable invocations

When a large SIMD value is broken down into pieces to invoke the Callable function, the size of that piece can be obtained from the Callable function invocation's parameter type but it can also be useful to pass in the index of that piece too. For example, suppose a function is invoking an intrinsic to perform a special type of memory store operation. Each register-sized sub-piece of the incoming SIMD needs to know its offset so that it can be written to the correct pointer offset. The following example code illustrates how this could happen:

```
auto special_memory_store(simd<float, 32> x, float* ptr)
{
    // Invoke the most appropriate intrinsic for the given simd types.
    auto do_native =
        [=]<typename T, typename ABI>(basic_simd<T, ABI> data, auto idx) {
            (_mm256_special_store_ps(ptr + idx, static_cast<__m256>(data)));
        };

    simd_invoke_indexed(do_native, x);
}
```

The `simd_invoke_indexed` function is now used instead, and this expects that the Callable will take an extra parameter giving the compile-time offset of the first index of the sub-SIMD value (i.e., 0, 8, 16 and 24 in this example).

Note in this example that the Callable does not return a value itself, nor does the call to `simd_invoke_indexed`.

3.1.1. Design option - probing the index capabilities

Rather than requiring the user to invoke an indexed callable using `simd_invoke_indexed` an alternative would be to allow `simd_invoke` to probe the Callable to see if it accepts one extra index parameter, and if it does to pass in that index to each Callable invocation. The advantage of doing this is to have a single function which can use the index or not depending on the Callable being used. The disadvantage is that the mechanism of probing the Callable may be fragile and lead to obscure errors.

3.2. Considerations in using `simd_invoke`

These are a set of considerations for using `simd_invoke`. In the following, SIMD-like will be taken to be a type that is either a `basic_simd`, or a `basic_simd_mask`, and whose native size can be queried.

All the arguments that are passed to the callable function must be `simd` or `simd_mask` values.

When more than one SIMD-like object is used as an argument, all of the SIMD-like objects must have the same size (i.e., you can't call `simd_invoke` with SIMD values which don't break down into pieces whose respective sizes are the same).

For the native block size to be deduced, all the SIMD-like values must have the same native size. This is to ensure that when the SIMD-like arguments are broken into pieces they will always map to the same respective sizes. If the size cannot be deduced like this then the user must explicitly supply the block size.

The order of invoking the Callable function for each sub-block is undefined.

When the Callable returns a value, it must return a SIMD-like object. This is because there is no way to take non-SIMD results from the Callable and merge them together, except by using `simd_concat`.

When the Callable returns a SIMD-like object, it need not have the same size as its input arguments. For example, the Callable function could perform an operation like extracting multiples of some index, where the results have to be concatenated back together.

When the Callable returns a SIMD-like object, every invocation of the callable must return a SIMD-like object with the same element type. This is to ensure that the results can be glued together.

4. Wording

4.1. Add new section [`simd.invoke`]

🔗 **`simd_invoke` function calling utility** [`simd.simd_invoke`]

```
template<std::size_t BlockSize = 0, typename Fn, typename... Args>
constexpr auto simd_invoke(Fn&& fn, Args&&... args);
```

```
template<std::size_t BlockSize = 0, typename Fn, typename... Args>
constexpr auto simd_invoke_indexed(Fn&& fn, Args&&... args);
```

Constraints:

- `sizeof...(Args) > 0`.
- Each argument in `Args...` is either `basic_simd` or `basic_simd_mask`.
- For every argument in `Args...`, `Args::size`'s must be identical.
- `BlockSize` is either non-zero, or every argument in `Args...` must have the same native size.
- The `Fn` return type is either `void`, or `basic_simd`, or `basic_simd_mask`.

Effects:

Let `split-size` be a `BlockSize` if that is non-zero, or native size, otherwise.

- Invokes `Fn` with `args` split by `split-size`. The number of invocations is `Args::size / split-size + (Args::size % split-size == 0 ? 0 : 1)`. The invocation order of `Fn` over split args is unspecified.
- If `Fn` returns non-void type, collects the all `Fn` invocation results and returns it as a single value.

Returns:

If the `Fn` returns non-void, concatenate every `Fn` call result using `simd_concat` and returns that value. Otherwise, returns `void`.

References

Informative References

[P1928R3]

Matthias Kretz. *Merge data-parallel types from the Parallelism TS 2*. 3 February 2023. URL: <https://wg21.link/p1928r3>

[P1928R4]

Matthias Kretz. [*std::simd - Merge data-parallel types from the Parallelism TS 2*](#). 19 May 2023. URL:
<https://wg21.link/p1928r4>